

华东师范大学数据科学与工程学院实验报告

课程名称: 编译原理	年级: 2024	上机实践时间: 2026.03.28
指导教师: 黄波	姓名: 孙育泉	
上机实践名称: Lab 2	学号: 10234900421	

I 实验任务

使用 flexc++ 实现一个针对 tiger 语言的 lexical scanner, 要求包含以下部分:

- Basic specifications
- Comments handling
- String handling
- Error handling

II 实验过程

II.1 具体实现

首先, 除了 ID, INT, STRING 等特殊的 token 以外, 我们只需要调用 adjust() 来进行正常读取, 然后返回对应的 Parser 即可.

lex

```
1 ", " {adjust(); return Parser::COMMA;}
2 ":" {adjust(); return Parser::COLON;}
3 ";" {adjust(); return Parser::SEMICOLON;}
4 "(" {adjust(); return Parser::LPAREN;}
5 ")" {adjust(); return Parser::RPAREN;}
6 "[" {adjust(); return Parser::LBRACK;}
7 "]" {adjust(); return Parser::RBRACK;}
8 "{" {adjust(); return Parser::LBRACE;}
9 "}" {adjust(); return Parser::RBRACE;}
10 "." {adjust(); return Parser::DOT;}
11 "+" {adjust(); return Parser::PLUS;}
12 "-" {adjust(); return Parser::MINUS;}
13 "*" {adjust(); return Parser::TIMES;}
14 "/" {adjust(); return Parser::DIVIDE;}
15 "=" {adjust(); return Parser::EQ;}
16 "<" {adjust(); return Parser::NEQ;}
17 "<" {adjust(); return Parser::LT;}
18 "<=" {adjust(); return Parser::LE;}
19 ">" {adjust(); return Parser::GT;}
20 ">=" {adjust(); return Parser::GE;}
21 "&" {adjust(); return Parser::AND;}
22 "|" {adjust(); return Parser::OR;}
23 ":=" {adjust(); return Parser::ASSIGN;}
24 "if" {adjust(); return Parser::IF;}
25 "then" {adjust(); return Parser::THEN;}
26 "else" {adjust(); return Parser::ELSE;}
27 "while" {adjust(); return Parser::WHILE;}
28 "for" {adjust(); return Parser::FOR;}
29 "to" {adjust(); return Parser::TO;}
```

```

30 "do" {adjust(); return Parser::DO;}
31 "let" {adjust(); return Parser::LET;}
32 "in" {adjust(); return Parser::IN;}
33 "end" {adjust(); return Parser::END;}
34 "of" {adjust(); return Parser::OF;}
35 "break" {adjust(); return Parser::BREAK;}
36 "nil" {adjust(); return Parser::NIL;}
37 "function" {adjust(); return Parser::FUNCTION;}
38 "var" {adjust(); return Parser::VAR;}
39 "type" {adjust(); return Parser::TYPE;}

```

接下来，对于数字，由于定义了 `digit [0-9]`，因此我们可以直接使用 `digit+` 来匹配一个或多个数字，并将其转换为整数返回：

```

1 {digit}+ {
2     adjust();
3     return Parser::INT;
4 }

```

lex

接下来，对于标识符，由于 Tiger 语言中可以允许数字、字母、下划线，但是要求字母开头，于是可以写出正则表达式：

```

1 {letter}({letter}|{digit}|_)* {
2     adjust();
3     return Parser::ID;
4 }

```

lex

接下来处理注释部分。在 Tiger 语言中，注释是可以嵌套的，因此我们需要使用独占状态 `COMMENT` 以及一个变量 `comment_level_` 来专门记录嵌套的层级。

当词法分析器遇到 `/*` 时，会初始化嵌套层级为 1，并进入 `COMMENT` 状态：

```

1 "/*" {
2     adjust();
3     // 记录注释起始位置
4     comment_level_ = 1;    // 初始化嵌套层级
5     begin(StartCondition_::COMMENT);
6 }

```

lex

在 `<COMMENT>` 状态内部，我们需要处理以下几种情况：

- **嵌套加深**：如果再次遇到 `/*`，则将 `comment_level_` 递增，嵌套层级加深。
- **嵌套变浅与退出**：如果遇到 `*/`，则 `comment_level_` 递减；当层级归零（`comment_level_ = 0`）时，说明最外层注释已闭合，此时退回 `INITIAL` 正常状态。
- **换行符处理**：遇到 `\n` 时，必须调用 `errormsg_→Newline()` 通知错误处理模块，以保证后续报错时的行号准确无误。
- **普通字符处理**：对于注释内的其他所有普通字符 `.`，仅调用 `adjustStr()` 忽略即可。
- **异常处理**：如果遇到了文件结束符 `<<EOF>>` 但注释依然没有闭合，则调用 `errormsg_→Error` 报错 “unclosed comment”，并强制退回初始状态。

lex

```

1 <COMMENT>{
2     "/*" {

```

```

3      adjustStr();          // 内部字符, 只移动指针
4      comment_level_++;    // 嵌套层级加深
5  }
6  "*/" {
7      adjustStr();
8      comment_level--;    // 嵌套层级变浅
9      if (comment_level_ == 0) {
10         begin(StartCondition_::INITIAL); // 退回正常状态
11     }
12 }
13 \n {
14     adjustStr();
15     errormsg_→Newline(); // 换行必须通知 errormsg_
16 }
17 . {
18     adjustStr();          // 忽略注释内的其他所有普通字符
19 }
20 <<EOF>> {
21     // 如果文件结束了但注释还没闭合, 要报错
22     adjustStr();
23     errormsg_→Error(errormsg_→tok_pos_, "unclosed comment");
24     begin(StartCondition_::INITIAL);
25 }
26 }

```

然后, 处理字符串部分.

字符串的处理同样依赖于独占状态 STR. 当遇到双引号 " 时, 首先调用 `string_buf_.clear()` 清空之前遗留的缓冲区, 并进入 STR 状态.

在 <STR> 状态中, 最为核心的是各种转义序列的解析:

- (1) **基础转义字符**: 例如 `\\n`, `\\t`, `\\\\`, `\\\"`, 匹配到后直接向 `string_buf_` 中追加对应的真实字符 `\n`, `\t`, `\\`, `\"`.
- (2) **三位十进制 ASCII 码 (`\\ddd`)**: 通过截取匹配字符串的后三位数字, 使用 `std::stoi` 将其转换为整数, 再通过 `static_cast<char>` 强制转换后追加到缓冲区.
- (3) **控制字符 (`\\^c`)**: 提取出匹配字符串中的字母, 利用位运算技巧 (与 31 进行按位与运算, 即 831) 直接将其转换为对应的控制字符 ASCII 码.
- (4) **多行字符串格式化**: Tiger 语言允许使用 `\\[ \\t\\n\\f]+\\` 来忽略多行字符串间的空白符. 此时需要遍历匹配到的字符串, 并在遇到换行符时调用 `errormsg_→Newline()` 以维持准确的行号.

对于字符串中的常规字符, 直接追加匹配到的首字符即可. 如果遇到未转义的闭合双引号 ", 则调用 `setMatched(string_buf_)` 将匹配的字面量替换为拼接好的真实字符串, 退出 STR 状态并返回 `Parser::STRING`.

如果在字符串中遇到了非法的原生换行符 `\\n`, 报错 "illegal newline in string"; 如果遇到 `<<EOF>>` 而字符串未闭合, 则报错 "unclosed string".

lex

```

1  /* 遇到双引号, 进入字符串状态 */
2  \" {
3      adjust();          // 记录字符串起始位置 (tok_pos_ 留在这里)
4      string_buf_.clear(); // 清空之前遗留的 buffer

```

```
5   begin(StartCondition_::STR);
6 }
7
8  /* 在 STR 状态下处理内部逻辑 */
9  <STR>{
10   \" {
11       adjustStr();
12       setMatched(string_buf_); // 核心: 把 flex 匹配到的字面量替换为你拼接的真实字符串
13       begin(StartCondition_::INITIAL);
14       return Parser::STRING;
15   }
16
17   /* 处理转义字符 */
18   \\n { adjustStr(); string_buf_ += '\n'; }
19   \\t { adjustStr(); string_buf_ += '\t'; }
20   \\\ { adjustStr(); string_buf_ += '\\'; }
21   \\\" { adjustStr(); string_buf_ += '\"'; }
22
23   /* 处理 \ddd (三位十进制 ASCII 码) */
24   \\[0-9]{3} {
25       adjustStr();
26       std::string text = matched();
27       // text 是类似 "\099" 的字符串. 我们要截取从索引 1 开始的 "099", 转成整数, 再强制
       转换成 char
28       int ascii_val = std::stoi(text.substr(1));
29       string_buf_ += static_cast<char>(ascii_val);
30   }
31
32   /* 处理 \\^c (控制字符) */
33   \\\"^[a-zA-Z] {
34       adjustStr();
35       char control_char = matched()[2]; // 取出匹配字符串 "\\^C" 里的第 3 个字符,
       即 'C'
36       // 一个非常优雅的位运算技巧: 大写字母减去 64 刚好就是控制字符的 ASCII 码, 或者直接与
       31 按位与
37       string_buf_ += static_cast<char>(control_char & 31);
38   }
39
40
41   \\[ \\t\\n\\f]+\\ {
42       adjustStr();
43       std::string matched_str = matched();
44       for (char c : matched_str) {
45           if (c == '\\n') {
46               errormsg_→Newline();
47           }
48       }
49   }
50
51   \\n {
52       adjustStr();
53       errormsg_→Error(errormsg_→tok_pos_, "illegal newline in string");
54       errormsg_→Newline();
55   }
56
57   /* 处理普通字符 */
```

```
58     . {
59         adjustStr();
60         string_buf_ += matched()[0];
61     }
62
63     <<EOF>> {
64         adjustStr();
65         errmsg_→Error(errmsg_→tok_pos_, "unclosed string");
66         begin(StartCondition_::INITIAL);
67     }
68 }
```

最后，在正常的 INITIAL 状态下，我们需要将空格和制表符 [\t]+ 过滤掉并调整指针位置。遇到换行符 \n 时，除了调整位置外，必须调用 errmsg_→Newline() 更新行号记录。

对于任何不匹配上述所有规则的非法输入，则直接通过 errmsg_→Error(errmsg_→tok_pos_, "illegal token") 抛出非法词法单元的错误。

II.2 遇到的问题

- (1) 第一次写的时候，忽略了变量的名字可以出现下划线，导致有些测试样例没有通过。
- (2) lex 文件写规则是有优先级的，第一次写的时候，把数字和标识符的规则写在了最前面，导致关键字都被错误地识别为普通的变量名。
- (3) 实现字符串的时候，没有实现控制字符。